

AI Usage Record

1. Type of Generative AI Used

- GitHub Copilot

2. Details of AI-Assisted Code

File: `MainPage.js`

Lines: 123-145

Prompt Used: "I want to implement a global search functionality for messages across all chatrooms. The search should display the message content and the chatroom it belongs to."

AI Response: The AI provided a solution using Firestore's `collectionGroup` query to search messages and retrieve their parent chatroom information. It suggested using `where` clauses for prefix matching and iterating over the results to fetch chatroom names.

Modified Code:

```
const handleSearch = async () => {
  if (!searchQuery.trim()) return;

  try {
    const q = query(
      collectionGroup(db, 'messages'),
      where('text', '>=', searchQuery),
      where('text', '<=', searchQuery + '\uf8ff')
    );
    const querySnapshot = await getDocs(q);

    const results = [];
    for (const doc of querySnapshot.docs) {
      const data = doc.data();
      const parentRef = doc.ref.parent.parent;
      if (parentRef) {
        const chatroomDoc = await getDoc(parentRef);
        const chatroomData = chatroomDoc.data();
        if (chatroomData.members.includes(currentUser.uid)) {
          const chatroomName = chatroomDoc.exists() ? chatroomData.name : 'Unknown';
          results.push({
            id: doc.id,
            text: data.text,
            chatroomName,
          });
        }
      }
    }
  }
}
```

```

    } else {
      console.warn('Parent reference is null for document:', doc.id);
    }
  }

  setSearchResults(results);
} catch (error) {
  console.error('Error searching messages:', error);
}
};

```

Explanation: This code implements a search functionality that ensures only messages from chatrooms the user belongs to are included in the results. The AI's suggestion was adapted to include a membership check for privacy.

File: `styles.css`

Lines: 200-220

Prompt Used: "Add dramatic animations for message entry and user profile picture hover effects. Make the animations visually striking."

AI Response: The AI suggested using CSS keyframes for animations. For message entry, it proposed a sliding and scaling effect. For user profile pictures, it recommended a 3D rotation and scaling effect on hover.

Modified Code:

```

@keyframes slideInDramatic {
  0% {
    transform: translateX(100%) scale(0.5);
    opacity: 0;
  }
  50% {
    transform: translateX(-10%) scale(1.2);
    opacity: 0.8;
  }
  100% {
    transform: translateX(0) scale(1);
    opacity: 1;
  }
}

.message {
  animation: slideInDramatic 1s cubic-bezier(0.68, -0.55, 0.27, 1.55);
}

.user-profile-picture {
  transition: transform 0.6s ease-in-out, box-shadow 0.6s ease-in-out;
}

.user-profile-picture:hover {

```

```
transform: perspective(800px) rotateY(360deg) rotateX(360deg) scale(1.2);
box-shadow: 0 20px 40px rgba(0, 0, 0, 0.5);
}
```

Explanation: The animations enhance the user experience by making interactions more dynamic. The message entry animation creates a dramatic sliding effect, while the profile picture hover effect adds a 3D transformation for visual appeal.

File: ChatroomPage.js

Lines: 10-50

Prompt Used: "Implement a chatroom page where users can send and receive messages, with support for blocking users and displaying notifications."

AI Response: The AI suggested using Firebase Firestore for real-time message updates and filtering out messages from blocked users. It also recommended requesting notification permissions and displaying notifications for new messages.

Modified Code:

```
useEffect(() => {
  if (!chatroomId || !currentUser) return;

  const q = query(
    collection(db, `chatrooms/${chatroomId}/messages`),
    orderBy('timestamp')
  );
  const unsubscribe = onSnapshot(q, (snapshot) => {
    const messagesData = snapshot.docs
      .map((doc) => ({ id: doc.id, ...doc.data() }))
      .filter((message) => !blockedUsers.includes(message.uid)); // Exclude messages from blocked users
    setMessages(messagesData);
  });

  return () => unsubscribe();
}, [chatroomId, currentUser, blockedUsers]);

useEffect(() => {
  if (Notification.permission === 'default') {
    Notification.requestPermission().then((permission) => {
      console.log('Notification permission status:', permission);
    });
  }
}, []);
```

Explanation: This code ensures real-time updates for messages in the chatroom while filtering out messages from blocked users. It also handles notification permissions for displaying new message alerts.

File: `UserProfileModal.js`

Lines: 20-70

Prompt Used: "Create a user profile modal where users can edit and save their profile information, including uploading a profile picture."

AI Response: The AI provided a structure for the modal, including input fields for user details and a file input for uploading a profile picture. It also suggested using Firebase Firestore to save the updated profile data.

Modified Code:

```
const handleSave = async () => {
  try {
    const userRef = doc(db, 'users', currentUser.uid);
    const profileData = {
      photoURL: profilePicture,
      displayName: userName,
      email,
      phoneNumber,
      address,
    };
    await setDoc(userRef, profileData, { merge: true });
    alert('Profile updated successfully!');
    onClose();
  } catch (error) {
    console.error('Error updating profile:', error);
    alert('Failed to update profile.');
  }
};

const handleFileChange = (e) => {
  const file = e.target.files[0];
  if (file) {
    const reader = new FileReader();
    reader.onload = () => {
      setProfilePicture(reader.result);
    };
    reader.readAsDataURL(file);
  }
};
```

Explanation: This code allows users to edit their profile information and upload a profile picture. The changes are saved to Firebase Firestore, and the profile picture is previewed before saving.

File: `firebase.js`

Lines: 1-20

Prompt Used: "Set up Firebase configuration and initialize Firebase services for authentication and Firestore."

AI Response: The AI provided the basic setup for Firebase, including initializing Firebase App, Firestore, and Authentication services. It also included a Google Auth provider for third-party authentication.

Modified Code:

```
import { initializeApp } from 'firebase/app';
import { getAuth, GoogleAuthProvider } from 'firebase/auth';
import { getFirestore } from 'firebase/firestore';

const firebaseConfig = {
  apiKey: 'YOUR_API_KEY',
  authDomain: 'YOUR_AUTH_DOMAIN',
  projectId: 'YOUR_PROJECT_ID',
  storageBucket: 'YOUR_STORAGE_BUCKET',
  messagingSenderId: 'YOUR_MESSAGING_SENDER_ID',
  appId: 'YOUR_APP_ID',
};

const app = initializeApp(firebaseConfig);
const auth = getAuth(app);
const db = getFirestore(app);
export const googleProvider = new GoogleAuthProvider();
export { auth, db };
```

Explanation: This code initializes Firebase services, including Firestore and Authentication, and sets up a Google Auth provider for third-party login functionality.